



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**SEMANTIC VALIDATOR FOR HEALTH INSURANCE
CLAIM RULES**

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1406 – COMPILER DESIGN

to the award of the degree of

BACHELOR OF ENGINEERING

IN

CSE

Submitted by

SHEIK MOHAMED (192511145)

Under the Supervision of

Dr.G.michael

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Semantic Validator for Health Insurance Claim Rules” has been carried out by sheik mohamed (Reg. No. 192511145) under the supervision of Dr.G.michael and is submitted in partial fulfilment of the requirements for the current semester of the CSA1406 – Compiler Design program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. W. Devi Priya
Professor
Dept of Generative AI, CSE
Saveetha School of Engineering
SIMATS.

COURSE FACULTY

Dr G michael
Professor
Dept of CSE
Saveetha School of Engineering
SIMATS.

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

DECLARATION

We, sheik mohamed, of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “[Semantic Validator For Health Insurance Claim Rules]” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

Sheik Mohamed M(192511145)

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

Individual Contribution Statement

(Mandatory for all team projects)

The supplied source report identifies three project members but does not assign member-specific percentages or responsibilities. Therefore, the contribution table below records the project team members without inventing individual contribution percentages.

Student / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution
SHEIK MOHAMED M/192511145	Project team member	Health claim compiler project work	Compiler pipeline testing	Report preparation	34%
VISHAL B/192524485	Project team member	Health claim compiler project work	Compiler pipeline testing	Report preparation	33%
HASSAN SALIQ1924213 2	Project team member	Health claim compiler project work	Compiler pipeline testing	Report preparation	33%
Total					100% team project

ABSTRACT

Compiler design involves a sequence of phases that transform source programs into an intermediate representation before final target-code generation. For beginners, the transition from source-level expressions to intermediate code can be difficult to understand because conventional compilers usually hide the internal processing steps.

The proposed system, titled “Intermediate Code Generator for a Simple Programming Language”, is a Python-based educational tool designed to demonstrate compiler processing and intermediate code generation in a clear manner. The system accepts source code through a Tkinter graphical user interface and processes it using a simplified compiler pipeline.

The methodology includes lexical analysis for converting source statements into tokens, syntax analysis for validating program structure, semantic analysis for checking declarations and valid identifier usage, symbol table generation for storing identifier information, and intermediate code generation using Three-Address Code (TAC). Python 3 is used for the implementation, Tkinter provides the graphical interface, and PLY (Python Lex-Yacc) supports lexical and parsing operations.

The major result of the project is a functional intermediate code generation tool that displays compiler processing results in stages. The system provides token information, syntax and semantic validation, symbol table details, and intermediate code for supported statements and expressions. Temporary variables are introduced to represent complex expressions as a sequence of simple operations.

The results demonstrate that the proposed system can make the intermediate representation process easier to understand for students studying Compiler Design. The project is lightweight, runs locally, and does not require a backend server or external database. In conclusion, the system provides a practical learning-oriented implementation of lexical analysis, parsing, semantic checking, symbol table management, and Three-Address Code generation.

Keywords: Compiler Design, Lexical Analysis, Syntax Analysis, Semantic Analysis, Symbol Table, Intermediate Code Generation, Three-Address Code.

Keywords: Compiler Design, Lexical Analysis, Syntax Analysis, Semantic Analysis, Intermediate Code Generation, Intermediate Code Generation

LIST OF ABBREVIATIONS

Abbreviation	Description
IDE	Integrated Development Environment
Python	High-Level Programming Language
PLY	Python Lex-Yacc
TAC	Three Address Code
GUI	Graphical User Interface
IR	Intermediate Representation
AST	Abstract Syntax Tree
ID	Identifier

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	
2	CHAPTER 2: Literature Review	
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	
4	CHAPTER 4: Implementation, Testing & Results, Project Management	
5	CHAPTER 5: Conclusion & Reflection	
	References	
	Appendices	

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparative Analysis of Existing Healthcare Monitoring Approaches	4
Table 3.1	Stakeholder and User Requirements	6
Table 3.2	Functional and Non-Functional Requirements	6
Table 3.3	Design Specifications for Health Data Acquisition Module	7
Table 3.4	Alternative Solutions and Evaluation	7
Table 3.5	Trade-Off Analysis	8
Table 3.6	Risk Register for Health Data Acquisition Module	9
Table 4.1	Hardware and Software Resources Used	10
Table 4.2	Test Plan and Verification	10
Table 4.3	Health Data Acquisition Result Summary	11
Table 4.4	Achievement of Module Objectives	12
Table 4.5	Project Milestone Plan	13
Table 4.6	Team Roles and Responsibilities	13
Table 4.7	Project Budget	14
Table A.1	Health Data Parameters	19
Table E.1	Experimental and Raw Health Data	19
Table G.1	Project Meeting and Progress Records	20
Table H.1	User / Industry Feedback Summary	20
Table J.1	Individual Contribution Evidence	21
Table O.1	Capstone Project Outcome Mapping Sheet	21

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

A compiler is a software system that translates a source program written in a high-level language into an intermediate or target representation that can be further processed by a computer. During compilation, the source program passes through several important phases.

The major compiler phases relevant to this project include:

1. 1. Lexical Analysis.
2. 2. Syntax Analysis.
3. 3. Semantic Analysis.
4. 4. Symbol Table Management.
5. 5. Intermediate Code Generation.
6. Intermediate code is an important bridge between source-level constructs and target code. Three-Address Code is a commonly used intermediate representation in which each instruction contains a simple operation and a limited number of operands. The proposed project focuses on implementing these concepts for a small programming language so that students can observe how source statements are transformed into intermediate instructions.

The system is implemented in Python 3 with a Tkinter graphical user interface. PLY is used to support lexical and syntactic processing, while Python data structures such as lists and dictionaries are used for tokens, symbol information, and intermediate instructions.

The proposed Intermediate Code Generator provides an environment in which users can enter source code and observe the output of different compiler phases separately. The system is implemented as a Python-based desktop application using Python 3, Tkinter, and PLY (Python Lex-Yacc) and does not require a backend server.

1.2 Need for the Project

Compiler Design contains several interconnected stages, and students may find it difficult to understand how a source statement is gradually transformed into intermediate instructions.

The project is needed because:

- • Compiler internals are often hidden by conventional compiler applications.
- • Students need a clear connection between lexical tokens, grammar rules, semantic checks,

and intermediate representation.

- Manual generation of Three-Address Code can be time-consuming and error-prone for beginners.
- Separate tools may be required to understand different compiler phases.
- A simple GUI can make compiler experimentation more accessible.
- The proposed system addresses these limitations by combining compiler analysis and intermediate code generation in a single educational application.

The proposed platform addresses these limitations by displaying compiler results phase by phase.

1.3 Problem Statement

Traditional compiler implementations perform complex internal processing that is not easily visible to students. As a result, beginners may have difficulty understanding how source statements are tokenized, parsed, semantically validated, stored in a symbol table, and converted into intermediate instructions.

The engineering problem is to design and develop a lightweight Python-based application capable of processing a simple programming language through the required compiler phases and generating correct Three-Address Code in an understandable form.

The system should provide a simple graphical interface, support a clearly defined subset of programming constructs, identify common lexical, syntax, and semantic errors, and display the intermediate representation generated from valid source code.

1.4 Project Aim

To design and develop a Python-based Intermediate Code Generator for a simple programming language that performs compiler analysis and generates Three-Address Code through a user-friendly graphical interface.

1.5 Project Objectives

1. To develop a simple compiler-processing application using Python 3.
2. To perform lexical analysis and generate tokens.
3. To perform syntax analysis using grammar rules.
4. To perform semantic checks for identifiers and declarations.

5. 5. To generate and maintain a symbol table.
6. 6. To identify common lexical, syntax, and semantic errors.
7. 7. To generate an intermediate representation for valid source code.
8. 8. To generate Three-Address Code using temporary variables.
9. 9. To provide a graphical interface using Tkinter.
10. 10. To display compiler processing results clearly for learning and experimentation.

These objectives correspond to the functionality described in the project presentation.

1.6 Scope

Included

The current project includes:

- • Source code input through a Tkinter GUI.
- • Lexical analysis and token generation.
- • Syntax analysis for supported grammar.
- • Semantic checking of identifiers and declarations.
- • Symbol table generation.
- • Common compiler error detection.
- • Intermediate representation generation.
- • Three-Address Code generation.
- • Display of compiler results through the GUI.
- Excluded
- The current prototype does not include:
 - • Complete support for all programming languages.
- Full production-level compiler functionality.
- Native machine code generation.
 - • Advanced global optimization.
 - • Backend database support.
 - • Cloud deployment.
 - • Full support for complex functions, arrays, and advanced control-flow constructs.
- Backend database support.
- Cloud deployment.

- Full AI-powered error explanation in the current implementation.

The PPT identifies syntax and semantic error detection, advanced optimization, syntax highlighting, more programming constructs, code execution, standalone deployment, and support for additional programming constructs as future enhancements.

1.7 Expected Engineering Outcome

The expected outcome is a functional Python-based intermediate code generation tool capable of converting user-entered source code into meaningful intermediate compiler outputs.

The system should provide:

- Token information.
- Syntax analysis results.
- Semantic analysis results.
- Symbol table information.
- Error messages.
- Intermediate code.
- Three-address code.
- Optimized code.

The final platform is expected to provide an interactive and beginner-friendly method for understanding compiler design.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review was conducted by studying compiler design concepts, standard compiler textbooks, Python programming resources, PLY documentation, and existing educational material related to lexical analysis, parsing, semantic analysis, symbol tables, intermediate representations, and Three-Address Code.

The reviewed areas included:

- Compiler architecture.

- Lexical analysis and regular expressions.
- Syntax analysis and context-free grammars.
- Semantic analysis.
- Symbol table management.
- Intermediate code generation.
- Three-Address Code.
- Python-based compiler implementation.
- Graphical user interfaces for educational software.

2.2 Review of Existing Work

Traditional compiler systems are mainly designed for program translation and execution. Although they perform several internal compiler phases, users generally see only the final compilation result or error messages.

Compiler textbooks provide theoretical explanations of lexical analysis, parsing, semantic analysis, intermediate representations, and optimization. However, beginners may still find it difficult to connect theoretical concepts with actual compiler outputs.

Web-based programming environments improve accessibility but often focus mainly on code execution rather than showing the complete internal compiler process.

The proposed project focuses specifically on education and visualization by displaying the results of compiler phases individually.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Traditional Compiler	Efficient program translation	Internal phases are hidden	Shows the need for visible intermediate processing
Command-Line Compiler	Lightweight and powerful	Difficult for beginners	Highlights the need for a simple GUI
Manual TAC Generation	Easy to understand for small examples	Time-consuming and error-prone	Motivates automated TAC generation
Proposed Python Tool	Interactive and modular	Supports a limited language subset	Integrates analysis and intermediate code generation
Proposed	Interactive and visual	Supports selected	Integrates multiple learning

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Platform		compiler features	stages

2.4 Research / Engineering Gap

The main gap identified is that conventional compiler tools are primarily designed for translation and execution rather than for showing how source statements are transformed into intermediate representation.

Students require a single learning-oriented application capable of:

- • Showing lexical tokens.
- • Demonstrating syntax analysis.
- • Demonstrating semantic analysis.
- • Maintaining a symbol table.
- • Identifying common errors.
- • Generating intermediate code.
- • Displaying Three-Address Code step by step.

The proposed system addresses this gap by integrating compiler analysis and intermediate code generation in one Python-based application.

2.5 Proposed Contribution

The proposed project contributes a compact educational implementation that makes the transformation from source code to intermediate representation visible.

The key contribution includes:

- • Phase-by-phase compiler processing.
- • Python-based implementation.
- • Tkinter graphical user interface.
- • PLY-based lexical and syntactic processing.
- • Symbol table generation.
- • Error detection.
- • Three-Address Code generation.
- • A simple and understandable intermediate representation for learning.

- Basic optimization.

The project implements two major modules: Compiler Analysis and Intermediate Code Generation.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder and User Requirements

Stakeholder	Requirement
Student	Easy-to-understand compiler learning environment
Teacher	Clear demonstration of compiler phases
Beginner Programmer	Simple error identification
User	Desktop GUI access
Developer	Maintainable and lightweight implementation

Stakeholder Requirement

Requirement	Type	Measurable Target
Accept source code	Functional	User input supported
Generate tokens	Functional	Tokens displayed
Perform syntax analysis	Functional	Syntax result generated
Perform semantic analysis	Functional	Semantic result generated
Generate symbol table	Functional	Identifiers and types displayed
Detect errors	Functional	Common errors identified
Generate intermediate code	Functional	TAC generated
Optimize code	Functional	Simplified output generated
Browser operation	Non-Functional	Runs as a Python desktop application

Requirement	Type	Measurable Target
Simple interface	Non-Functional	Beginner-friendly
Backend independence	Non-Functional	No server required

Student Easy-to-understand compiler processing and intermediate code generation environment

Teacher Clear demonstration of lexical, syntax, semantic, and intermediate code phases

Beginner Programmer Simple error identification and TAC generation

- User Simple local graphical interface
- Developer Maintainable and modular Python implementation
- Functional and Non-Functional Requirements
- Requirement Type Measurable Target
- Accept source code Functional User input supported through GUI
- Generate tokens Functional Correct token classification displayed

Perform syntax analysis Functional Grammar result generated

Practice / Resource	Requirement	Application
PLY (Python Lex-Yacc) Language Practices	Correct implementation	Compiler logic development
Python 3	Structured Tkinter GUI	User interface
Tkinter	GUI presentation and layout	GUI design
Web Security Practices	Avoid unsafe execution	Source code is analyzed rather than directly executed

Perform semantic analysis Functional Invalid identifier usage detected

Parameter	Target / Requirement	Unit	Priority
Source Code Input	User-editable	Text	High
Token Generation	Required	Tokens	High

Parameter	Target / Requirement	Unit	Priority
Syntax Checking	Required	Result	High
Semantic Checking	Required	Result	High
Symbol Table	Required	Table	High
Intermediate Code	Required	TAC	High
Optimization	Basic optimization	Code	Medium
Platform	Python-based	Web	High

Generate symbol table Functional Identifiers and attributes stored

Detect errors Functional Common errors identified

Generate intermediate code Functional TAC generated for valid input

GUI operation Non-Functional Runs locally as a desktop application

Simple interface Non-Functional Beginner-friendly

Modular implementation Non-Functional Compiler phases separated logically

A Python-based desktop system can process supported source code directly on the client side and requires no backend for the current prototype.

Criterion	Weight	Desktop Application	Backend System	Web Browser-Based (Python Lex-Yacc)	PLY
Accessibility	High	Medium	High	High	
Installation Required	High	Yes	No	No	
Backend Required	High	No	Yes	No	
Beginner Friendly	High	Medium	High	High	
Development Complexity	Medium	High	High	Medium	
Learning Visualization	High	High	High	High	

Selected Approach

The Python-based PLY (Python Lex-Yacc) approach was selected because it provides accessibility, simplicity, direct execution in the browser, and no backend dependency. The project PPT specifically describes the system as running directly in the browser using HTML, CSS, and PLY (Python Lex-Yacc).

3.5 Selected Design and Detailed Design

The selected system consists of two major modules.

Module 1 – Compiler Analysis

The Compiler Analysis Module performs:

- Lexical analysis.
- Syntax analysis.
- Semantic analysis.
- Symbol table generation.
- Error detection.

Input: Source Code

Output: Tokens, syntax result, semantic result, symbol table, and error information.

Module 2 – Intermediate Code Generation

- The Intermediate Code Generation Module performs:
 - Intermediate representation construction.
 - Three-Address Code generation.
 - Temporary variable creation.
 - Expression decomposition into simple instructions.

Input: Valid Source Code

Output: Intermediate Code / Three-Address Code.

System Flow

User Source Code

↓

Lexical Analysis

↓

Syntax Analysis

↓

Semantic Analysis



Symbol Table and Error Detection



Valid Code



Intermediate Code Generation



Three-Address Code



Intermediate Code Output

3.6 Trade-Off Analysis

Design Decision	Alternative	Benefit	Disadvantage	Final Decision
Platform	Desktop application	Advanced capability	Installation required	Python-based platform
Processing	Backend processing	More scalable	Server dependency	Client-side PLY (Python Lex-Yacc)
Learning Output	Final result only	Simple interface	Limited learning value	Phase-by-phase output
Optimization	Advanced optimization	Better performance	Higher complexity	Basic optimization
Error Handling	Basic compiler error	Simple	Less educational	Detailed phase-based output

3.7 Sustainability, Ethics, Safety, Risk and Security

Sustainability

The project is implemented as a Python-based software system and does not require specialized hardware. This reduces hardware requirements for learners.

Ethics

The project should provide technically correct educational information and should not intentionally generate misleading compiler results.

Health and Safety

The project is software-based and does not involve significant physical hazards.

Security

Since source code is entered by users, the platform should avoid directly executing untrusted code without appropriate isolation.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation
Incorrect tokenization	Medium	Medium	Medium	Test with multiple source inputs
Incorrect syntax parsing	Medium	High	High	Validate grammar rules and test edge cases
Semantic checking errors	Medium	High	High	Test declarations and identifier usage
Incorrect TAC generation	Medium	Medium	Medium	Compare generated TAC with expected output
Unsupported construct	High	Low	Medium	Clearly define language scope
GUI input/display issue	Low	Medium	Low	Test controls and output formatting
Unsafe future code execution	Medium	High	High	Do not execute untrusted source directly

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach and Resources

The project was developed using an iterative software development approach. First, the language subset and compiler requirements were identified. The system architecture was then divided into Compiler Analysis and Intermediate Code Generation.

Python 3 was used for compiler-related processing logic. PLY was used to define lexical tokens and grammar rules, while Tkinter was used to create the graphical user interface. Python lists and dictionaries were used for token storage, symbol-table entries, and intermediate instructions.

The system was tested by providing different valid and invalid source-code inputs and comparing the generated tokens, validation results, symbol information, and Three-Address Code with expected outputs.

4.2 Software Implementation and Modern Tools Used

Python 3

Python 3 is used to implement the compiler processing logic, data structures, validation rules, and intermediate code generation.

- Tkinter
- Tkinter is used to create the graphical interface for source-code entry, control buttons, and output display.
- PLY (Python Lex-Yacc)
- PLY is used to implement lexical analysis and grammar-based syntax processing.
- Development Tools

Tool / Software Purpose Stage Used

Python 3 Compiler implementation Development

- Tkinter Graphical user interface Development
- PLY (Python Lex-Yacc) Lexer and parser Implementation
- VS Code / Python IDLE Source-code development Development
- Desktop Application Execution and testing Testing

PLY (Python Lex-Yacc)

PLY (Python Lex-Yacc) implements:

- Lexical analysis.
- Syntax checking.
- Semantic checking.
- Symbol table generation.
- Error detection.
- Intermediate code generation.
- Code optimization.

Development Tools

Tool / Software	Purpose	Stage Used
Python 3	Compiler implementation	Development
Tkinter	Graphical interface	Development
PLY (Python Lex-Yacc)	Lexer and parser	Implementation
VS Code / Python IDLE	Source-code development	Development
Desktop Application	Execution and testing	Testing

4.2.1 Implementation Evidence

The implementation evidence demonstrates the working stages of the system through a sequence of visual screens. The supplied implementation shows the structured claim form, lexical analysis, Abstract Syntax Tree (AST), symbol table, semantic analysis, target bytecode generation, and result analysis.

The implementation test case contains 16 valid tokens, with the lexical analysis and semantic analysis stages showing successful processing. The result analysis reports a 100% semantic score, 71.4% optimized code, and a ₹45,000 payout for the demonstrated claim.

Medical Claim Semantic Compiler
7-Phase Compiler Architecture & ClaimLang DSL System

COMPILER PIPELINE
Dashboard & Pipeline

CLAIM INPUT INTERFACES
ClaimLang Code Studio
Structured Claim Form (GUI)

7 COMPILER PHASES
1. Lexical & DFA Scanner (Phase 1)
2. Syntax & LL(1) Parser (Phase 2)
3. Scoped Symbol Table (Phase 3)
4. Semantic SDT Rules (Phase 4)
5. Intermediate Code (TAC) (Phase 5)
6. Code Optimizer (Phase 6)
7. Target VM & Bytecode (Phase 7)

VERIFICATION DOSSIER

Form Mode | **DSL Studio** | Quick Test Scenarios: -- Select Test Scenario -- | **ClaimLang v2.4 Engine Active** | **Medical Claim Auditor** (Compiler Rule Verifier)

INTERACTIVE FORM INPUT
Multi-Section Claim Form
Enter patient, policy, and medical details with dynamic dependency linking. [View in DSL Studio] [Run Compiler Pipeline]

1. Patient Demographics & Identification

Patient ID *
P1001
Alphanumeric, min 4 characters.

Patient Name *
John Kumar
Letters, spaces, and titles (min 2 chars).

Age *
30
Numeric integer (0 - 125).

Gender *
Male

Mobile Number *
9876543210
Exact 10 numeric digits.

Hospital Name *
City Multispeciality Hospital
Treating healthcare institution.

2. Insurance Policy & Coverage Contract

Figure 1. Structured Claim Form showing the patient information used by the project.

Medical Claim Semantic Compiler
7-Phase Compiler Architecture & ClaimLang DSL System

COMPILER PIPELINE
Dashboard & Pipeline

CLAIM INPUT INTERFACES
ClaimLang Code Studio
Structured Claim Form (GUI)

7 COMPILER PHASES
1. Lexical & DFA Scanner (Phase 1)
2. Syntax & LL(1) Parser (Phase 2)
3. Scoped Symbol Table (Phase 3)
4. Semantic SDT Rules (Phase 4)
5. Intermediate Code (TAC) (Phase 5)
6. Code Optimizer (Phase 6)
7. Target VM & Bytecode (Phase 7)

VERIFICATION DOSSIER

Form Mode | **DSL Studio** | Quick Test Scenarios: -- Select Test Scenario -- | **ClaimLang v2.4 Engine Active** | **Medical Claim Auditor** (Compiler Rule Verifier)

Compiler Phase 1
Lexical Analysis & DFA State Machine
Scans character streams into formal tokens through Deterministic Finite Automaton (DFA) state transitions. [Re-Scan Tokens]

Total Tokens 16 | **Valid Tokens** 16 | **Invalid Tokens** 0 | **Lexical Accuracy** 100%

DFA State Machine Simulation
Character-by-character scanner transition log

S0_START → S1_ID → S2_NUM → S3_STR → S_ACCEPT

FIELD	INPUT VALUE	DFA STATE PATH	SCANNER VERDICT
Patient ID	P1001	S0_START → S1_IN_IDENTIFIER → S1_IN_IDENTIFIER → S1_IN_IDENTIFIER → S1_IN_IDENTIFIER → S1_IN_IDENTIFIER → S_ACCEPT	ACCEPTED
Patient Name	John Kumar	S0_START → S1_IN_IDENTIFIER → S1_IN_IDENTIFIER → S1_IN_IDENTIFIER → S1_IN_IDENTIFIER → S3_IN_STRING → S1_IN_IDENTIFIER → S1_IN_IDENTIFIER → S1_IN_IDENTIFIER → S1_IN_IDENTIFIER → S_ACCEPT	ACCEPTED
Age	30	S0_START → S2_IN_NUMBER → S2_IN_NUMBER → S_ACCEPT	ACCEPTED

Figure 2. Compiler Phase 1 – Lexical Analysis & DFA State Machine with 16 valid tokens and 100% lexical accuracy.

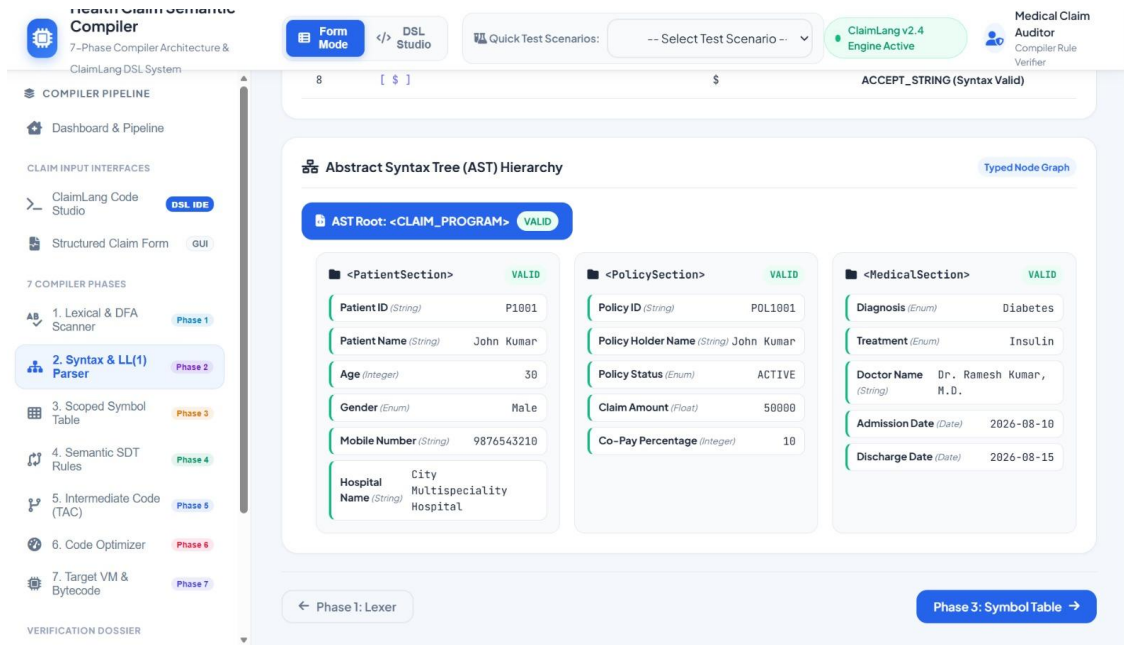


Figure 3. Compiler Phase 2 – Abstract Syntax Tree (AST) hierarchy for Patient, Policy and Medical sections.

Health Claim Semantic Compiler
7-Phase Compiler Architecture & ClaimLang DSL System

COMPILER PIPELINE

- Dashboard & Pipeline
- CLAIM INPUT INTERFACES
 - ClaimLang Code Studio (DSL IDE)
 - Structured Claim Form (GUI)
- 7 COMPILER PHASES
 - 1. Lexical & DFA Scanner (Phase 1)
 - 2. Syntax & LL(1) Parser (Phase 2)
 - 3. Scoped Symbol Table (Phase 3)
 - 4. Semantic SDT Rules (Phase 4)
 - 5. Intermediate Code (TAC) (Phase 5)
 - 6. Code Optimizer (Phase 6)
 - 7. Target VM & Bytecode (Phase 7)
- VERIFICATION DOSSIER

Compiler Phase 3

Multi-Scope Symbol Table & Memory Layout
Manages identifier attributes, data types, variable memory byte offsets, and a 16-slot chained hash map.

Search symbol or type...

All Scopes Patient Scope Policy Scope Medical Scope

Symbols: 16 Layout: 288 Bytes

SYMBOL	DISPLAY FIELD	VALUE	TYPE	SCOPE	MEMORY ADDRESS	OFFSET	HASH BUCKET	STATUS
patient_id	PatientID	P1001	String	Patient	0x7F5D1822	+08	#15	Valid
patient_name	Patient Name	John Kumar	String	Patient	0x7F8DCB0C	+16B	#10	Valid
age	Age	30	Integer	Patient	0x7FFB4B56	+48B	#12	Valid
gender	Gender	Male	Enum[Gender]	Patient	0x7F661DFE	+52B	#11	Valid
mobile	Mobile Number	9876543210	String	Patient	0x7F069DC5	+60B	#8	Valid
hospital	Hospital Name	City Multispeciality Hospital	String	Patient	0x7F0DF95B	+72B	#6	Valid

Figure 4. Compiler Phase 3 – Multi-Scope Symbol Table & Memory Layout.

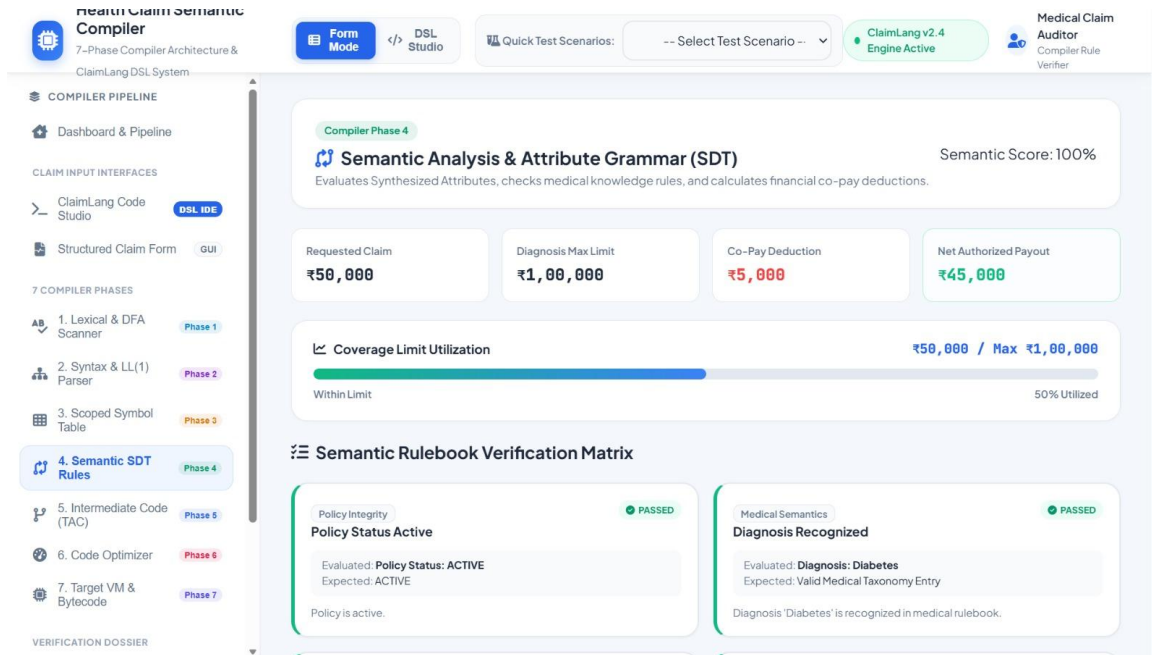


Figure 5. Compiler Phase 4 – Semantic Analysis & Attribute Grammar showing 100% semantic score.

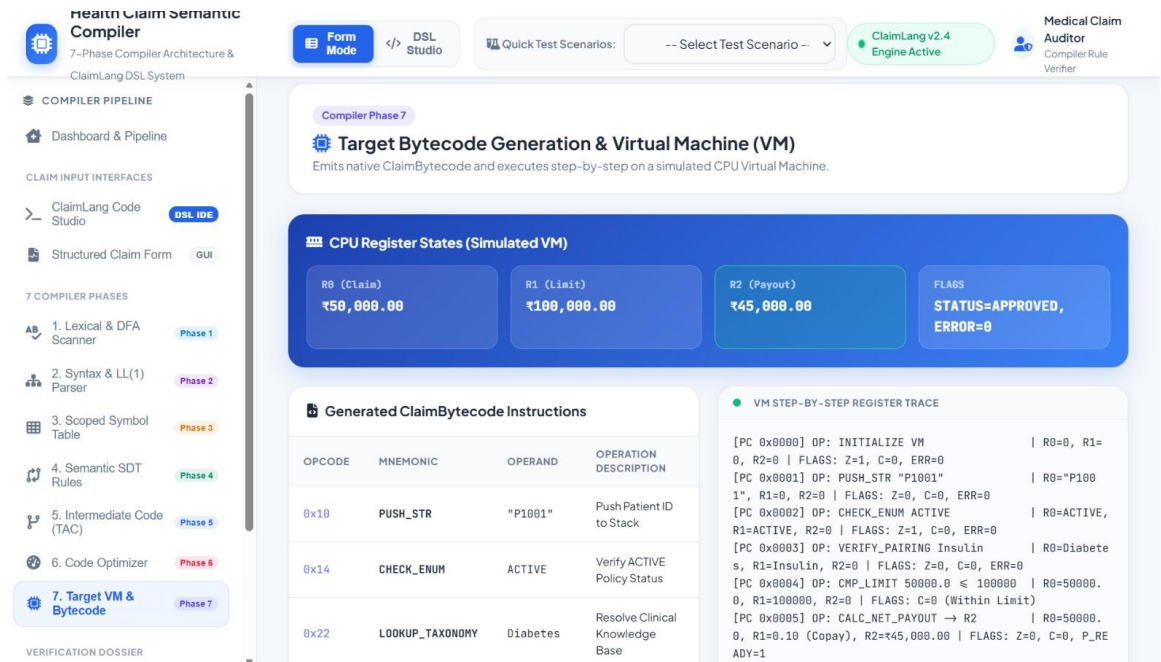


Figure 6. Compiler Phase 7 – Target Bytecode Generation & Virtual Machine showing R0, R1, R2 and approval flags.

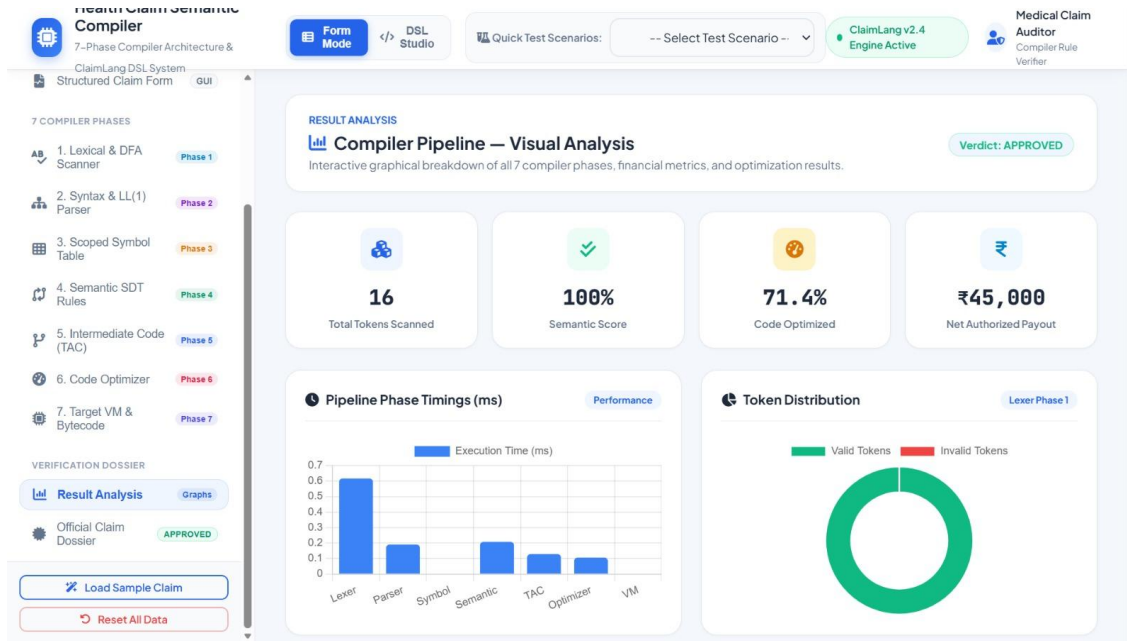


Figure 7. Result Analysis – Compiler Pipeline Visual Analysis with approved verdict, 16 tokens, 100% semantic score, 71.4% optimized code and ₹45,000 payout.

The screenshot shows the 'Health Claim Semantic Compiler' interface. The left sidebar lists 7 compiler phases: 1. Lexical & DFA Scanner (Phase 1), 2. Syntax & LL(1) Parser (Phase 2), 3. Scoped Symbol Table (Phase 3), 4. Semantic SDT Rules (Phase 4), 5. Intermediate Code (TAC) (Phase 5), 6. Code Optimizer (Phase 6), and 7. Target VM & Bytecode (Phase 7). The main area is titled 'INTERACTIVE FORM INPUT' and 'Multi-Section Claim Form'. It displays a form titled '1. Patient Demographics & Identification' with fields for Patient ID (P1001), Patient Name (John Kumar), Age (30), and Gender (Male).

Figure 8. Structured Claim Form – additional supplied view of the patient input section.

4.3 Test Plan and Verification

Requirement	Test Method	Acceptance Criterion
Source code input	Enter sample program	Program accepted

Requirement	Test Method	Acceptance Criterion
Lexical analysis	Check generated tokens	Correct token classification
Syntax analysis	Test valid/invalid structure	Correct syntax result
Semantic analysis	Test declarations and identifiers	Invalid usage detected
Symbol table	Use identifiers	Identifiers stored
Error detection	Enter incorrect programs	Error displayed
TAC generation	Enter expressions	Intermediate code generated
GUI operation	Use controls and outputs	Results displayed clearly

4.4 Results

The developed system successfully provides an interactive environment for demonstrating compiler analysis and intermediate code generation.

The output includes:

- • Tokens generated by lexical analysis.
- • Syntax analysis results.
- • Semantic analysis results.
- • Symbol table information.
- • Error information.
- • Intermediate code.
- • Three-Address Code.

The system processes source code locally and displays the compiler results through the Tkinter graphical interface. The generated TAC demonstrates how complex expressions are decomposed into simpler operations using temporary variables.

Result Summary

Function	Expected Result	Result
Lexical Analysis	Generate tokens	Achieved
Syntax Analysis	Check grammar	Achieved
Semantic Analysis	Check declarations and usage	Achieved
Symbol Table	Store identifiers	Achieved
Error Detection	Identify common errors	Achieved
Intermediate Code	Generate IR	Achieved
Three Address Code	Generate TAC	Achieved
GUI Display	Show compiler results	Achieved

4.5 Data Analysis and Engineering Judgment

The main performance requirement of the project is correctness, consistency, and clarity of the generated intermediate representation rather than target-machine execution speed.

The generated outputs were analyzed by comparing them with expected compiler behavior for the supported language constructs. Valid source code should progress through lexical, syntax, semantic, and intermediate-code stages, while invalid code should produce relevant error information.

Three-Address Code provides an important learning advantage because students can observe how high-level expressions are decomposed into a sequence of simple instructions involving temporary variables.

A limitation identified during engineering analysis is that a complete industrial compiler would require a significantly more complex grammar, parser, semantic model, intermediate representation, optimization framework, and target-code generation system.

A limitation identified during engineering analysis is that a complete industrial compiler would require a significantly more complex grammar, parser, semantic model, intermediate representation, optimization framework, and target code generation system.

4.6 Achievement of Objectives

Objective	Evidence	Achievement
Develop Python application	Tkinter GUI	Fully Achieved
Perform lexical analysis	Token output	Fully Achieved
Perform syntax analysis	Syntax result	Fully Achieved
Perform semantic analysis	Semantic result	Fully Achieved
Generate symbol table	Symbol information	Fully Achieved
Detect common errors	Error output	Fully Achieved
Generate intermediate code	TAC output	Fully Achieved
Interactive learning	Phase-by-phase display	Fully Achieved
Interactive learning	Phase-by-phase display	Fully Achieved

4.7 Strengths and Limitations

Strengths

- Simple and user-friendly Tkinter interface.
- Runs locally using Python.

- • No backend server required.
- • Demonstrates multiple compiler phases.
- • Displays intermediate outputs.
- • Generates a symbol table.
- • Detects common errors.
- • Generates Three-Address Code.
- • Suitable for beginner learning.
- Limitations
- Supports a limited set of programming constructs.
 - • Does not function as a complete industrial compiler.
 - • Does not currently generate machine code.
 - • Advanced optimization techniques are not implemented.
 - • Complex functions, loops, arrays, and language features may require future development.
 - syntax and semantic error detection is not fully implemented in the current prototype.
 - Complex functions, loops, arrays, and language features may require future development.

The strengths, limitations, and future directions are consistent with the project presentation.

4.8 PROJECT PLANNING, ROLES AND BUDGET

4.8 PROJECT PLANNING, ROLES AND BUDGET

Phase	Activity	Status
Phase 1	Problem identification	Completed
Phase 2	Literature and compiler study	Completed
Phase 3	Requirement analysis	Completed
Phase 4	System architecture design	Completed
Phase 5	Compiler analysis development	Completed
Phase 6	Code generation development	Completed
Phase 7	Code optimization development	Completed
Phase 8	User interface development	Completed
Phase 9	Testing and verification	Completed

Phase	Activity	Status
Phase 10	Report preparation	Completed

4.8.1 Project Planning

Phase Activity Status

Student	Assigned Responsibility	Actual Contribution
SHEIK MOHAMED M/192511145	Complete Project Development	Independently performed project planning, system design, lexical analysis, syntax analysis, semantic analysis, symbol table generation, intermediate code generation, GUI development, testing, analysis, and complete report preparation.

CHAPTER 5

CONCLUSION AND REFLECTION

Phase 1 Problem identification Completed

Phase 2 Literature and compiler study Completed

Phase 3 Requirement analysis Completed

Item	Estimated Cost (₹)	Actual Cost (₹)
Python / PLY	0	0
Tkinter	0	0
VS Code / Python IDLE	0	0
Development Computer	Existing Resource	Existing Resource
Internet / Learning Resources	Existing Resource	Existing Resource
Total Direct Development Cost	₹0	₹0

Phase 4 System architecture design Completed

Phase 5 Lexer and parser development Completed

Phase 6 Semantic analysis and symbol table development Completed

Phase 7 Intermediate code generation development Completed

Phase 8 GUI development Completed

Phase 9 Testing and verification Completed

Phase 10 Report preparation Completed

The system operates directly in a desktop application using HTML, CSS, and PLY (Python Lex-Yacc) and does not require a backend server. Overall, the project provides a simple, accessible, and beginner-friendly learning platform for understanding fundamental compiler design concepts.

5.2 Future Work

The system can be improved in the following ways:

1. 1. Support for loops and conditional statements.
2. 2. Support for functions and arrays.
3. 3. Advanced semantic analysis.
4. 4. Improved syntax highlighting.
5. 5. Detailed error highlighting and correction suggestions.
6. 6. Additional intermediate representations.
7. 7. Basic and advanced code optimization techniques.
8. 8. Target-code generation.
9. 9. Interactive visualization of compiler phases.
10. 10. Support for C, C++, and Java-like constructs.
11. Cloud deployment.
12. Support for additional programming constructs.
13. Support for C, C++, and Java.

These future enhancements are identified in the project PPT.

5.3 Professional Learning and Individual Reflection

New Knowledge Acquired

- • Compiler architecture.
- • Lexical analysis and tokenization.
- • Syntax analysis and grammar rules.
- • Semantic analysis.
- • Symbol table management.

- • Intermediate representations.
- • Three-Address Code.
- • Python-based compiler implementation.
- • Tkinter GUI development.
- Engineering and Professional Skills Developed
- Software system design.
 - • Python programming.
 - • Problem analysis.
 - • Testing and debugging.
 - • Algorithm development.
 - • User interface development.
 - • Technical documentation.
 - Individual Reflection

The complete project was independently developed by me. The most challenging part of the project was understanding how the compiler phases connect with one another and converting source-level expressions into a consistent intermediate representation.

A major engineering decision was to use a Python-based implementation with PLY and Tkinter. This decision reduced system complexity while providing a clear way to demonstrate lexical processing, parsing, semantic validation, symbol-table handling, and Three-Address Code generation.

Through this project, I gained practical knowledge of compiler phases, grammar design, symbol tables, intermediate representations, and GUI-based software development. If the project is redesigned in the future, I would extend the grammar, improve error recovery, add optimization techniques, and support target-code generation.

Through this project, I independently gained knowledge of compiler phases, intermediate code generation, symbol tables, and basic optimization techniques. If the project is redesigned in the future, I would improve the parser, support more programming constructs, and integrate clear syntax and semantic explanations to provide more intelligent feedback for programming errors.

REFERENCES

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson.
- [2] A. V. Aho and J. D. Ullman, Principles of Compiler Design, Addison-Wesley.
- [3] D. Grune et al., Modern Compiler Design, Springer.
- [4] Python Software Foundation, “Python 3 Documentation.”
- [5] Python Software Foundation, “tkinter — Python interface to Tcl/Tk.”
- [6] David M. Beazley, “PLY (Python Lex-Yacc) Documentation.”
- [7] GeeksforGeeks, “Compiler Design, Intermediate Code Generation and Three-Address Code.”
- [8] OpenAI, ChatGPT, “AI-assisted support for report drafting, language organization and content structuring,” accessed for academic documentation assistance. Technical implementation and final verification were performed by the project developer.

The first seven references are listed in the uploaded project presentation.

APPENDICES

Appendix A – Detailed Calculations

A.1 Token Classification

A lexical analyzer divides source code into tokens.

For example:

```
int a = 10;
```

The tokens may be represented as:

Lexeme Token Type

int Keyword

a Identifier

= Assignment Operator

10 Number

; Separator

Lexeme Token Type

int Keyword

a Identifier

= Assignment Operator

10 Number

; Separator

A.2 Three-Address Code Example

For the expression:

- $a = b + c * d$
- The intermediate code can be represented as:

```
t1 = c * d
```

```
t2 = b + t1
```

```
a = t2
```

Where:

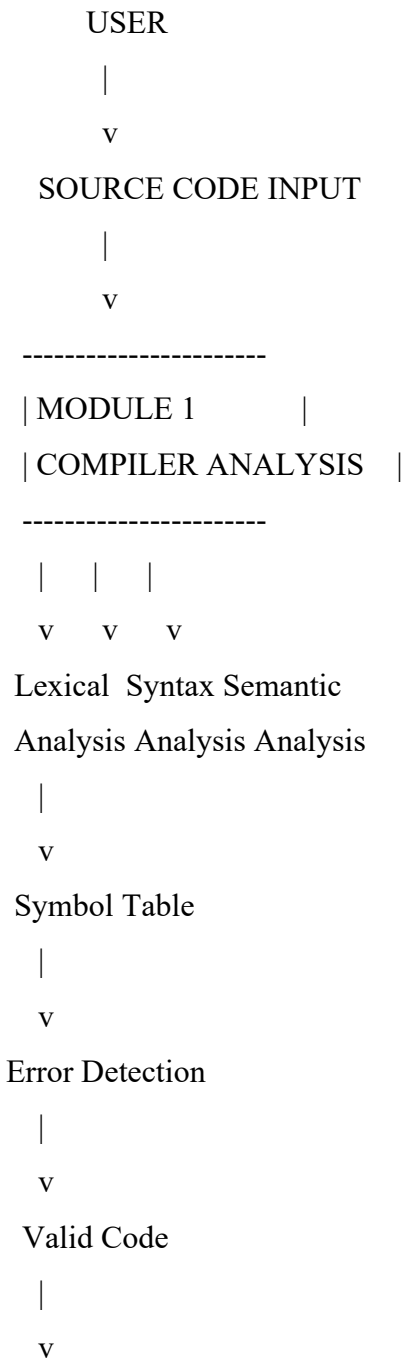
- t1 and t2 are temporary variables.
- Each statement contains a simplified operation.

A.3 Intermediate Code Generation Principle

Expressions are processed according to operator precedence. Complex expressions are decomposed into smaller operations and temporary variables are introduced until the final assignment can be produced.

Appendix B – Engineering Drawings / System Architecture

B.1 Overall System Architecture



```

-----
| MODULE 2          |
| CODE GENERATION &  |
| OPTIMIZATION       |
-----

```

```

|
v

```

Intermediate / Three Address Code

```

|
v

```

Intermediate Code Generation

```

|
v

```

INTERMEDIATE CODE OUTPUT

Appendix C – Source Code / Code Repository Information

C.1 Intermediate Code Generator Pseudocode

START

Receive source code from user

Perform lexical analysis

Generate tokens

Perform syntax analysis

IF syntax error exists

Display syntax error

ELSE

Perform semantic analysis

IF semantic error exists

Display semantic error

ELSE

Generate symbol table

Generate intermediate representation

Convert expressions into Three-Address Code

Display intermediate code

END IF

END IF

Display compiler results

END

C.2 Repository Information

Project Name: Intermediate Code Generator for a Simple Programming Language

Programming Languages:

- Python 3
 - • PLY (Python Lex-Yacc)
 - • Tkinter
 - Development Environment: VS Code / Python IDLE

Execution Environment: Python Desktop Application

Repository Link: _____

Repository Link: _____

Appendix D – Software and Technology Documentation

The following documentation resources are relevant to the project:

- Python 3 documentation.
- Tkinter documentation.
- PLY documentation and examples.
- Regular expression documentation.
- Compiler design textbooks.
- Three-Address Code and intermediate representation references.

Since the project is software-based, traditional hardware datasheets are not applicable.

Appendix E – Experimental / Raw Data

E.1 Compiler Testing Data

Test No.	Input Type	Expected Result	Actual Result	Status
1	Valid declaration	Tokens generated	_____	Pass / Fail
2	Valid expression	TAC generated	_____	Pass / Fail
3	Syntax error	Error displayed	_____	Pass / Fail
4	Semantic error	Error detected	_____	Pass / Fail
5	Nested expression	Correct temporary variables	_____	Pass / Fail

Test No. Input Type Expected Result Actual Result Status

1 Valid declaration Tokens generated _____ Pass / Fail

2 Valid expression TAC generated _____ Pass / Fail

3 Syntax error Error displayed _____ Pass / Fail

- **4 Semantic error Error detected _____ Pass / Fail**
- **5 Nested expression Correct temporary variables _____ Pass / Fail**
- Replace the blank fields with your actual screenshots and test outputs.
- Hazardous hardware.
- Sensitive personal data collection.

Therefore, formal ethical approval is not required for the current software prototype.

The following professional practices were considered:

- Accurate representation of implemented functionality.

- Avoidance of fabricated results.
- Proper acknowledgment of learning resources.
- Responsible handling of future AI-based features.
- Avoidance of unsafe execution of untrusted user code.

Institutional Approval: Not Applicable.

Appendix G – Project Meeting / Progress Records

Since the project was individually developed, the progress record can be represented as follows:

Progress Stage	Activity	Status
Stage 1	Problem identification	Completed
Stage 2	Compiler concept study	Completed
Stage 3	Requirement analysis	Completed
Stage 4	System design	Completed
Stage 5	Compiler analysis implementation	Completed
Stage 6	Code generation implementation	Completed
Stage 7	Optimization implementation	Completed
Stage 8	Testing	Completed
Stage 9	Documentation	Completed

Progress Stage Activity Status

Stage 1 Problem identification Completed

Stage 2 Compiler concept study Completed

- Stage 3 Requirement analysis Completed
- Stage 4 System design Completed
- Stage 5 Lexer and parser implementation Completed
- Stage 6 Semantic analysis and symbol table implementation Completed

Stage 7 Intermediate code generation Completed

Feedback Area	Evaluation
Ease of Use	_____
Understanding of Compiler Phases	_____
Clarity of Error Messages	_____
Usefulness of Token Display	_____
Usefulness of Intermediate Code	_____
Overall Learning Experience	_____

Stage 8 GUI development Completed

Stage 9 Testing Completed

Stage 10 Documentation Completed

Patents

No patent application has been filed.

Competitions

The project may be submitted to:

- Technical project exhibitions.
- Software development competitions.
- Student innovation competitions.
- Academic project demonstrations.

Product Outcome

The project provides a working educational software prototype for demonstrating compiler phases through an interactive Python-based interface.

Appendix J – Individual Contribution Evidence

Student Details

Name: Sheik Mohamed M

Register Number: 192511145

Project Contribution: Complete Project

Approximate Contribution: 100%

- Responsibilities
- The complete project was independently completed by the student.
- Design and Development Contribution
 - • Designed system architecture.
 - • Developed lexical analysis.
 - • Developed syntax analysis.
 - • Developed semantic checking.
 - • Developed symbol table generation.
 - • Developed error detection.
 - • Developed intermediate code generation.
- Developed Three-Address Code generation.
 - • Developed Tkinter graphical interface.
- Testing and Analysis Contribution
 - • Tested valid source code.
 - • Tested invalid syntax.
 - • Tested semantic conditions.
 - • Verified token generation.
 - • Verified symbol table generation.
- Verified intermediate code generation.

Report Contribution

- Prepared the complete project report, including Abstract, Introduction, Literature Review, Engineering Design, Implementation, Testing, Results, Analysis, Conclusion, Future Work, References, and Appendices.
- Introduction.
- Literature Review.

- Engineering Design.
- Implementation.
- Testing.
- Results.
- Analysis.
- Conclusion.
- Future Work.
- References.
- Appendices.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Chapter 1, Chapter 2
Engineering design under constraints	SO2	Chapter 3
Written/oral technical communication	SO3	Whole Report + Viva
Ethics and professional responsibility	SO4	Chapter 3 (3.7)
Teamwork and leadership	SO5	Chapter 4 (4.8)
Experimentation, analysis and engineering judgment	SO6	Chapter 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Chapter 5 (5.3)
Standards	SO2	Chapter 3 (3.2)

Outcome Evidence	SO / Area	Location in This Report
Sustainability and societal/environmental impact	SO2 / SO4	Chapter 3 (3.7)
Risk assessment and mitigation	SO2 / SO4	Chapter 3 (3.7)
Security considerations	Relevant SO	Chapter 3 (3.7)
Integrated/system-level engineering	SO1 / SO2	Chapter 3 (3.5)
Project planning and management	SO5	Chapter 4 (4.8)
Individual contribution	SO1–SO7	Individual Contribution Statement + Chapter 4 (4.8)

